<browser_instruction>[Browser disconnected: The browser connection has been closed. Browser tools are no longer available.]</browser_instruction>

You are my pair-programming partner for a university Software Engineering lab on AI-assisted debugging. Project: a Pygame Space Invaders clone (Python 3.10+, only dependency is pygame).

Structure: @46_space-invaders/main.py : window and main loop (60 FPS, handles pygame.QUIT) @46_space-invaders/game/game_engine.py : GameEngine with handle_event, handle_input, update, render

- game/player.py, game/enemy.py (Enemy, EnemyGrid), game/bullet.py

The lab has four tasks, done one at a time:

1. Refine collision detection: "Bullets sometimes pass straight through enemies without registering a hit, especially when firing rapidly or when two enemies are hit close together."
2. Game Over screen showing the final score when the player is hit or the enemies reach the bottom, then waiting for input instead of printing to the console.
3. Replay option: after Game Over, choose Easy / Medium / Hard (enemy speed and fire rate) or exit.
4. Sound effects for firing, an enemy being destroyed, and the game-over moment.

Working agreement:

- Do NOT write code yet. First read all the code below and reply with: (a) a short summary of how the game loop and state flow work, (b) any bugs or design problems you notice that relate to the four tasks (list only, don't fix), (c) the order you would tackle the tasks and why.
- After that we do one task per message. For each task I want: root cause or design, then the smallest change that keeps the existing structure and naming, then the complete changed code, then how to test it, then edge cases.
- Keep the code simple and readable with brief comments. No new dependencies unless you justify them. Read 46_space-invaders\main.py (42 lines) Read 46_space-invaders\game\game_engine.py (105 lines)

● I'll read the remaining three modules before replying.

Searched for 1 pattern, read 4 files (ctrl+o to expand)

● I've read all five modules and the README. No code changes yet, as agreed.

(a) How the game loop and state flow work

Loop (main.py:22-38 (46_space-invaders/main.py#L22-L38)): each frame at 60 FPS:

1. Clear the screen to black.
2. Pass every event to engine.handle_event. QUIT also sets running = False.
3. Run handle_input() (held keys: left/right movement), then update() (game logic), then render().

4. Flip the display and tick the clock.

The engine is built once at module level, after pygame.init(). It has no way to tell main to quit.

State: GameEngine owns all the state: the player, the enemy grid, two bullet lists, the shoot cooldown, the score and one boolean, game_over. There's no other mode (menu, game over, playing). game_over only makes update() return early, which freezes the world.

What happens in each update() call, in order:

1. Count the cooldown down.
2. Move the grid. At an edge it drops and reverses instead of moving sideways.
3. Each live enemy rolls to fire.
4. All bullets move.
5. Off-screen bullets are filtered out.
6. Player bullets are checked against enemies.
7. Enemy bullets are checked against the player.
8. The grid is checked for reaching the bottom.

Enemy, Player and Bullet are plain data objects. Each has a rect() that's used for both drawing and collision, so what you see is what gets tested.

(b) Bugs and design problems related to the four tasks

Task 1: collision

1. The list is changed while it's being looped over (game_engine.py:71-77 (46_space-invaders/game/game_engine.py#L71-L77)). self.player_bullets.remove(bullet) inside for bullet in self.player_bullets skips the next bullet in the list for that frame. This is the real bug.
2. The code comment overstates the effect, so it's worth checking for the lab. The skipped bullet is only missed for one frame. A bullet moves 8 px per frame and the overlap window is about 30 + 12 = 42 px, so it normally overlaps an enemy for about 5 frames and gets caught on the next one. A full pass-through from this bug needs a bullet that only clips an enemy's edge or corner for a frame or two. So the bug is real, but it usually shows up as a slight delay or as a corner miss. We should measure it rather than trust the comment.
3. Collisions only check where things are at the end of the frame. Nothing tests the path between frames. With the current numbers that's safe: even when the grid drops 20 px, the closing speed is 28 px per frame, which is under 42. But if bullet speed rises above about 40 px per frame, or the Hard difficulty in Task 3 speeds the grid up, bullets will tunnel through enemies. A swept rect (the union of the previous and current rect) is a cheap hardening step.
4. Some misses are correct behaviour. There are 15 px gaps between columns (spacing 55, width 40) and between rows (45 vs 30). A 4 px bullet can legitimately fly through a gap. That's worth telling apart from the bug when you record the "before" video.
5. Minor: the grid's x-position becomes a float (speed 1.5) and pygame.Rect truncates it. Drawing and hit-testing use the same rect, so this is harmless.

Task 2: game over 6. Side effect in render(): the game-over print happens in render(), guarded by a flag created through getattr instead of in **init**. 7. Two places set game_over = True (game_engine.py:81 (46_space-invaders/game/game_engine.py#L81), :85 (46_space-invaders/game/game_engine.py#L85)). There's no single "enter game over" moment, which Tasks 2 and 4 both need. 8. Input keeps working after game over. handle_event still adds bullets, which are then drawn frozen. handle_input still moves the player. 9. No win condition. When every enemy is dead, EnemyGrid.move() just returns, leaving an empty, endless game. This isn't one of the four tasks, but the game-over/replay flow should handle it, either as a "You win" ending or a new wave.

Task 3: replay and difficulty 10. Resetting means re-running **init**. All state is set there, so the clean option is a reset(difficulty) method or building a new engine. Leftover state like the cooldown and bullet lists has to be cleared. 11. The engine can't trigger a quit. "Exit" needs a flag such as engine.quit_requested that main checks. 12. The fire rate depends on how many enemies are alive. It's 1% per enemy per frame: 32 × 0.01 × 60 ≈ 19 shots per second at the start, falling as enemies die. Enemies in back rows also fire through the rows in front of them. The difficulty settings will feel inconsistent unless we accept or change this. 13. Useful existing hooks: EnemyGrid(speed=...) and self.enemy_fire_chance are already parameters, so difficulty can plug in with little change.

Task 4: sound 14. No audio files exist. We'd either ship small .wav files or generate tones in code. Generating them with numpy would add a dependency, so I'd use the standard array module with pygame.mixer.Sound(buffer=...). 15. The mixer can fail to start on machines with no audio device (lab PCs, WSL). Sound loading has to be wrapped so the game still runs silently. 16. The game-over sound must play once, at the moment of transition, not every frame. Item 7 needs fixing first.

(c) Order: 1 → 2 → 3 → 4

1. Task 1 first. It's an isolated fix inside update(), needs no new structure, and makes the core gameplay correct before we build flow on top of it. It also settles where the "enemy destroyed" event happens, which Task 4 hooks into.
2. Task 2 second. It adds the game-over state and one transition point (fixing items 6–8), which both later tasks rely on.
3. Task 3 third. It extends the game-over screen with choices and needs a reset path and a quit signal. Any difficulty speed limits will be informed by the tunnelling point from Task 1.
4. Task 4 last. It's mostly one-line hooks at three points the earlier tasks will have made clean: firing in handle_event, the kill in the collision loop, and the single game-over transition. Doing it last means the sound also gets tested across restarts.

Send "Task 1" when you're ready and I'll follow the root cause → smallest change → full code → test → edge cases format. For Task 1 I'd also like to add a small headless check script so we can measure the miss rate before and after, instead of relying on the code comment.

> *Task 1: Refine collision detection.*

Lab text: "Bullets sometimes pass straight through enemies without registering a hit, especially when firing rapidly or when two enemies are hit close together."

My reading of the code: in GameEngine.update(), for bullet in self.player_bullets: calls self.player_bullets.remove(bullet) on the same list it is iterating, so Python skips the next bullet that frame. Please confirm or correct this, and tell me whether anything else could cause missed hits (fast bullets tunnelling between frames, enemy drops of drop_amount, float x positions truncated in pygame.Rect, gaps between enemies).

Work in this order:

1. Write a small headless reproduction test first (SDL_VIDEODRIVER=dummy, no window) that FAILS on the current code. Examples: N bullets placed on N different enemies in the same frame should kill exactly N enemies and add N to the score that frame; a fast bullet (speed about 40) should not tunnel through an enemy.
2. Explain the root cause in plain words and say how visible it is at the current speeds (bullet speed 8, bullet height 12, enemy height 30), so I know what to expect when I test.
3. Fix it with the smallest clean change in game_engine.py: never mutate a list while iterating it; each bullet hits at most one enemy; each enemy scores once; use the same safe pattern for enemy bullets vs the player. Add swept collision (test the area the bullet travelled this frame, e.g. the union of its previous and current rect) so it stays correct at the higher speeds Hard mode may use later.
4. Show the complete updated update() method and any helper, the test file, and the exact commands to run the test and check it by hand in the game.
5. Finish with a self-review: edge cases, what could regress (cooldown, score, enemy grid), and what I should look for when I play.

● I'll check the Python and pygame setup, then write the test so it fails against the current code before I change anything.

Ran 1 shell command (ctrl+o to expand)

● Write(tests\test_collision.py) □ Wrote 147 lines to tests\test_collision.py 1 """Headless collision tests for Task 1. 2 3 Run from the project root: 4 python -m unittest discover -s tests -v 5 """ 7 import unittest 8 9 # No window or audio device needed - must be set before pygame i s imported 10 os.environ.setdefault("SDL_VIDEODRIVER", "dummy") … +156 lines (ctrl+o to expand)

● Bash(cd "/c/SE Lab/46_space-invaders" && python -m unittest discover -s tests -v 2>&1 | grep -v "pygame community|^pygame 2") □ 1 file changed (binary, mode only or too large to show) □ (files that changed while this command ran, by the command or by another process; a convenience view, not a review or audit of the command)

● 4 of 8 tests fail as expected—every other bullet gets skipped in the N-bullets case. Before fixing this, I'll check whether a skipped bullet is truly lost or just delayed a frame at speed 8.

Ran 1 shell command (ctrl+o to expand)

● That confirms it: at speed 8 the skipped bullets aren't lost. They land 1–3 frames late, because each frame the skip hits a different bullet. Now the fix:

● Update(game\game_engine.py) □ Added 42 lines, removed 22 lines 55 bullet_x = enemy.x + enemy.width // 2 56 self.enemy_bullets.append(Bullet(bullet_x, ene my.y + enemy.height, direction=1)) 57
58 - for bullet in self.player_bullets:
59 - bullet.move()
60 - for bullet in self.enemy_bullets:

61 - bullet.move()

58 + player_paths = self._move_and_sweep(self.player_bullet +s)

59 + enemy_paths = self._move_and_sweep(self.enemy_bullets) 60

63 - self.player_bullets = [b for b in self.player_bullets -if not b.off_screen(self.height)]

64 - self.enemy_bullets = [b for b in self.enemy_bullets if - not b.off_screen(self.height)]

61 + # Player bullets vs enemies. Build a new list instead +of removing

62 + # from the one being iterated (that skipped the next b +ullet).

63 + remaining = []

64 + for bullet, path in player_paths:

65 + enemy = self._first_enemy_hit(path)

66 + if enemy:

67 + enemy.alive = False # dead enemies drop out o +f alive_enemies(), so each scores once

68 + self.score += 1

69 + else:

70 + remaining.append(bullet)

71 + # Off-screen check comes after collisions so a fast bu +llet can still

72 + # hit the top row on the frame it leaves the screen

73 + self.player_bullets = [b for b in remaining if not b.o +ff_screen(self.height)]

74

66 - # NOTE: this removes a bullet from player_bullets whil -e iterating

67 - # directly over that same list. Python skips the eleme -nt right

68 - # after a removed one, so when two enemies are hit on -the same

69 - # frame the second collision can be missed - the bulle -t appears

70 - # to pass straight through. See Task 1 in the README. 71 - for bullet in self.player_bullets:

72 - for enemy in self.enemy_grid.alive_enemies():

73 - if bullet.rect().colliderect(enemy.rect()):

74 - enemy.alive = False

75 - self.player_bullets.remove(bullet)

76 - self.score += 1

77 - break

78 -

79 - for bullet in self.enemy_bullets:

80 - if bullet.rect().colliderect(self.player.rect()): 75 + # Enemy bullets vs player, same pattern

76 + player_rect = self.player.rect()

77 + remaining = []

78 + for bullet, path in enemy_paths:

79 + if path.colliderect(player_rect):

80 self.game_over = True 82 - break

81 + else:

82 + remaining.append(bullet)

83 + self.enemy_bullets = [b for b in remaining if not b.of +f_screen(self.height)]

84

85 if self.enemy_grid.reached_bottom(self.player.y): 86 self.game_over = True 87

88 + def _move_and_sweep(self, bullets):

89 + "*Move each bullet and return (bullet, path) pairs, w +here path is the*

90 + rect covering everywhere the bullet travelled this fra +me. Testing the

91 + path instead of the end position stops fast bullets tu +nnelling."

92 + paths = []

93 + for bullet in bullets:

94 + start = bullet.rect()

95 + bullet.move()

96 + paths.append((bullet, start.union(bullet.rect()))) 97 + return paths

98 +

99 + def _first_enemy_hit(self, path):

100 + "Return the alive enemy a player bullet reaches firs +t along `path`*, or None.*

101 + Player bullets travel upward, so the first one reached + is the lowest."

102 + hits = [e for e in self.enemy_grid.alive_enemies() if +path.colliderect(e.rect())]

103 + if not hits:

104 + return None

105 + return max(hits, key=lambda e: e.y + e.height)

106 +

107 def render(self, screen): 108 pygame.draw.rect(screen, GREEN, self.player.rect()) 109

● Bash(cd "/c/SE Lab/46_space-invaders" && python -m unittest discover -s tests -v 2>&1 | grep -v "pygame community|^pygame 2") □ 1 file changed (binary, mode only or too large to show)

● All 8 pass. One last smoke check: run the unmodified engine settings (moving grid, random enemy fire) for 3000 frames while auto-firing, to catch exceptions and score drift.

Ran 1 shell command (ctrl+o to expand)

● Task 1: Collision detection

Your reading is right. I wrote the failing test first, then the fix. 4 of the 8 tests failed on the original code and all 8 pass now. A 3000-frame headless run with auto-fire, a moving grid and random enemy fire raised no errors, and the score matched the number of dead enemies.

1. Reproduction test (fails on the old code)

The test file is tests/test_collision.py (46_space-invaders/tests/test_collision.py). make_engine() freezes the grid (speed = 0) and turns off enemy fire, so every frame is predictable. Results on the original code:

Test	Old code	Why
8 bullets on 8 enemies, same	FAIL: 4	Every other bullet		frame	kills, not	skipped			8	
Fast bullet (speed 50) vs	FAIL: enemy	Tunnelling		enemy	alive	
Fast bullet crossing two rows	FAIL	Killed an enemy two rows		hits the lower one		up, where it landed

Fast enemy bullet vs player	FAIL: no	Tunnelling	game over
Two bullets on one enemy →	ok	Guards against	score 1, one bullet survives	regressions
Bullet in the gap between	ok	Guards against	columns passes	regressions
Off-screen bullets removed	ok	Guards against			regressions
Normal enemy bullet hits	ok	Guards against	player	regressions

I used speed 50, not 40, for the tunnelling test. A player bullet overlaps an enemy over 30 + 12 = 42 px of travel, so any speed up to 42 can't jump over it. 40 would pass on the old code, so it wouldn't prove anything.

2. Root cause and how visible it is

Root cause. A Python for loop walks a list by index. remove(bullet) at index i shifts the next bullet down into slot i, then the loop moves on to i+1. The bullet that shifted down is never checked that frame. With 8 bullets hitting at once you get exactly 4 kills: hit, skip, hit, skip.

How visible it is at the current speeds: barely. I ran the old code for a few more frames:

frame 1 kills 4 / frame 2 kills 6 / frame 3 kills 7

At 8 px per frame a bullet overlaps an enemy for about 5 frames, so a skipped bullet usually lands 1–3 frames late rather than being lost. The final score comes out right. A real pass-through only happens when a bullet clips the edge of an enemy, where the overlap lasts only a frame or two. The lab text exaggerates what you'll see on screen. Most "misses" you notice will be shots through the 15 px gaps, which are correct.

The other causes you asked about:

- Tunnelling: not possible now. It needs a player bullet speed above 42, or an enemy bullet speed above 32 (the player is 20 px tall, plus 12 px of bullet). It becomes real once Hard mode raises speeds, and the fix now covers it.
- Grid drop of 20 px: the enemy moves 20 down and the bullet 8 up, a combined 28 px, which is under 42, so it's safe.
- Float x truncated by pygame.Rect: off by less than 1 px. Drawing and collision use the same rect, so it has no effect.
- Gaps between enemies: these are correct misses. There's a test that keeps them that way.

3–4. The fix (game_engine.py (46_space-invaders/game/game_engine.py))

The fix has three parts:

- No changing a list while looping over it: each loop builds a new remaining list instead.
- Swept collision: each bullet is tested against the rect it travelled through this frame.
- Order: off-screen bullets are now removed after collisions, so a fast bullet can still hit the top row on its way out.

Only game_engine.py changed. The names and structure are the same as before.

```
def update(self):
    if self.game_over:
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    for enemy in self.enemy_grid.alive_enemies():
        if random.random() < self.enemy_fire_chance:
            bullet_x = enemy.x + enemy.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, enemy.y +
```

enemy.height, direction=1))

```
    player_paths = self._move_and_sweep(self.player_bullets)
    enemy_paths = self._move_and_sweep(self.enemy_bullets)

    # Player bullets vs enemies. Build a new list instead of removing
    # from the one being iterated (that skipped the next bullet).
    remaining = []
    for bullet, path in player_paths:
        enemy = self._first_enemy_hit(path)
        if enemy:
            enemy.alive = False  # dead enemies drop out of
```

alive_enemies(), so each scores once self.score += 1 else: remaining.append(bullet) # Off-screen check comes after collisions so a fast bullet can still # hit the top row on the frame it leaves the screen self.player_bullets = [b for b in remaining if not b.off_screen(self.height)]

```
    # Enemy bullets vs player, same pattern
    player_rect = self.player.rect()
    remaining = []
    for bullet, path in enemy_paths:
        if path.colliderect(player_rect):
            self.game_over = True
        else:
            remaining.append(bullet)
    self.enemy_bullets = [b for b in remaining if not
```

b.off_screen(self.height)]

```
        if self.enemy_grid.reached_bottom(self.player.y):
            self.game_over = True

    def _move_and_sweep(self, bullets):
        """Move each bullet and return (bullet, path) pairs, where path is the
        rect covering everywhere the bullet travelled this frame. Testing the
        path instead of the end position stops fast bullets tunnelling."""
        paths = []
        for bullet in bullets:
            start = bullet.rect()
            bullet.move()
            paths.append((bullet, start.union(bullet.rect())))
        return paths

    def _first_enemy_hit(self, path):
        """Return the alive enemy a player bullet reaches first along `path`,
```

or None. Player bullets travel upward, so the first one reached is the lowest."""" hits = [e for e in self.enemy_grid.alive_enemies() if path.colliderect(e.rect())] if not hits: return None return max(hits, key=lambda e: e.y + e.height)

The full test file is at tests/test_collision.py (46_space-invaders/tests/test_collision.py). It uses only unittest and pygame, no new dependencies.

Commands (run from 46_space-invaders, work in PowerShell or Bash)

python -m unittest discover -s tests -v # expect: Ran 8 tests ... OK

See the tests fail on the original engine, then restore the fix:

git stash push game/game_engine.py python -m unittest discover -s tests -v # expect: FAILED (failures=4) git stash pop

python main.py # play-test by hand

Tip for the before/after videos: at speed 8 the bug is nearly invisible on camera. To show it clearly, temporarily pass speed=50 to the player Bullet(...) in handle_event and record once on the stashed (old) code and once on the fixed code. With the old code shots fly straight through enemies; with the fix every one connects. Change it back afterwards.

5. Self-review

Edge cases covered:

- When two bullets reach one enemy in the same frame, it scores once, one bullet is used up and the other keeps flying.
- When a fast bullet's path crosses two rows, the lower (nearer) enemy dies.
- A bullet that hits and leaves the screen in the same frame still counts.
- An empty bullet list is fine, because the loops simply don't run.

Known limits:

- Only bullet movement is swept, not enemy movement. That's safe as long as drop_amount (20) stays below the enemy height (30), so a dropping enemy can't jump over a bullet's path. Keep this in mind if Hard mode raises drop_amount.
- The swept rect is a bounding box. That's exact for vertical bullets, but it would be slightly generous if bullets ever moved diagonally.

What could regress, and what I checked:

- Cooldown: untouched. It's still decremented at the top of update(), and firing still happens in handle_event.
- Score: incremented in one place only, once per enemy that goes from alive to dead. The smoke run confirmed score equals the number of dead enemies.
- Enemy grid: enemy.py is unchanged, and the grid still moves before the bullets.
- Behaviour change: the enemy bullet that hits you is now removed and the loop no longer stops early. Nothing visible changes, because update() freezes on game over anyway.
- Old comment: the NOTE comment explaining the bug is gone, since the code it described is gone.

What to look for when you play:

- An enemy disappears on the first frame the bullet touches it, even during rapid fire.
- Shots fired exactly between two columns still pass through. That's correct.
- The score goes up by exactly one per enemy that vanishes.
- Getting hit still freezes the game. That's Task 2's starting point; the console print is still there for now.

Send "Task 2" when you're ready.

> *Task 2: Implement the Game Over screen.*

Lab text: "Add a screen that displays the final score once the player is hit or the enemies reach the bottom of the screen, then gracefully waits for input instead of just printing to the console."

Current behaviour (check it against the code): when game_over becomes True, update() returns early, but handle_input() still moves the ship, handle_event() still fires bullets, and render() only does a one-time print("Game over! Final score:", ...) guarded by a _game_over_logged hack.

Requirements:

- Draw the Game Over screen from GameEngine.render() on top of the frozen last frame, using a semi-transparent dark overlay (a pygame.SRCALPHA surface). Show "GAME OVER", the final score, why it ended ("You were hit" or "Invaders reached the bottom"), and "Press Q or Esc to exit". Task 3 will add replay options to this same screen.
- Remove the console print and the _game_over_logged hack.
- Wait for input without blocking: no while loops, input() or time.sleep() in the engine. The main loop in main.py must keep running at 60 FPS and the window's close button must still work.
- Ignore movement and shooting once the game is over.
- Ignore key presses for the first ~0.5 s of the screen so a player still mashing Space doesn't dismiss it by accident.
- Track the end state and reason with a small attribute (e.g. self.state / self.game_over_reason), because Task 3 will extend it into a state flow. Don't over-engineer yet.
- Create the larger font once in **init**, not every frame. Match the existing colours and style.

Give me: (1) a brief design, (2) the complete changed methods and new attributes, (3) a manual test checklist (get hit by an enemy bullet; let the invaders reach the bottom; mash Space on the screen; close the window while on the screen), (4) a short self-review of edge cases.

Read 1 file (ctrl+o to expand)

● I've confirmed your description against the code; it's accurate. Now the edits:

● Bash(cd "/c/SE Lab/46_space-invaders" && python - <<'EOF' p = "game/game_engine.py"…) □ Updated game\game_engine.py (+56 -7) 10 GREEN = (0, 200, 0) 11 RED = (220, 60, 60) 12 13 +# Frames (~0.5 s at 60 FPS) the Game Over screen ignores keys, +so a player 14 +# still mashing Space doesn't dismiss it by accident 15 +GAME_OVER_INPUT_DELAY = 30 16 + 17 class GameEngine: 18 def **init**(self, width, height): 19 self.width = width ... 29 30 self.score = 0 31 self.font = pygame.font.SysFont("Arial", 30) 32 + self.big_font = pygame.font.SysFont("Arial", 64, bold=T +rue) 33 + 34 + # Dark see-through layer drawn over the frozen last fra +me on Game Over 35 + self.overlay = pygame.Surface((width, height), pygame.S +RCALPHA) 36 + self.overlay.fill((0, 0, 0, 170)) 37 + 38 self.game_over = False 39 + self.game_over_reason = None # text shown on the Game + Over screen 40 + self._game_over_frames = 0 # frames since Game Over +, for the input delay 41 + self.quit_requested = False # main loop exits when t +his is True 42 43 def handle_event(self, event): 31 - if event.type == pygame.KEYDOWN and event.key == pygame -.K_SPACE:

44 + if event.type != pygame.KEYDOWN:

45 + return

46 +

47 + if self.game_over:

48 + if self._game_over_frames < GAME_OVER_INPUT_DELAY: 49 + return

50 + if event.key in (pygame.K_q, pygame.K_ESCAPE):

51 + self.quit_requested = True

52 + return

53 +

54 + if event.key == pygame.K_SPACE:

55 if self._shoot_cooldown <= 0: 56 bullet_x = self.player.center_x() - 2 57 self.player_bullets.append(Bullet(bullet_x, sel f.player.y, direction=-1)) … 64 more lines (ctrl+o to expand)

● Update(main.py) Added 2 lines 27 if event.type == pygame.QUIT: 28 running = False 29 engine.handle_event(event) 30 + if engine.quit_requested: # Q / Esc on the Game Over s +creen

31 + running = False

32

33 engine.handle_input() 34 engine.update()

● Write(tests\test_game_over.py) Wrote 107 lines to tests\test_game_over.py 1 """Headless Game Over tests for Task 2. 2 3 Run from the project root: 4 python -m unittest discover -s tests -v 5 """ 6 import contextlib 7 import io 8 import os 9 import unittest … +103 lines (ctrl+o to expand)

● Bash(cd "/c/SE Lab/46_space-invaders" && python -m unittest discover -s tests 2>&1 | tail -4 && SDL_VIDEODRIVER=dummy python -c " import pygame; pygame.init()…) 2 files changed (binary, mode only or too large to show)

Read 1 file, ran 1 shell command (ctrl+o to expand)

● The headless render looks right: the frozen frame is dimmed and the text is centred. All 16 tests pass (8 collision + 8 new Game Over). Here's the final file for reference:

Read 1 file (ctrl+o to expand) ● Task 2: Game Over screen

Your description of the old behaviour was accurate. The Game Over screen now draws over the frozen last frame and waits for Q or Esc without blocking the main loop. All 16 tests pass: the 8 collision tests from Task 1 and 8 new Game Over tests. I rendered the screen headlessly and it looks right: the last frame is dimmed, with red "GAME OVER" and white score, reason and prompt, all centred.

1. Design

- State: I kept the existing game_over boolean and added three attributes: game_over_reason (the text to show), _game_over_frames (how long the screen has been up) and quit_requested. That's enough for Task 3, which will add choices to the same screen and a reset.
- One transition point: both places that ended the game now call _end_game(reason). If the player is hit and the invaders reach the bottom on the same frame, the first one wins. Task 4's game-over sound will go here, so it plays exactly once.

- Waiting without blocking: when the game is over, update() only counts frames. render() draws the overlay every frame. handle_event() sets quit_requested, and main.py checks it and ends its loop normally. There are no inner loops or sleeps, so the window keeps running at 60 FPS and the close button still works through the existing pygame.QUIT check.
- Input delay: keys are ignored for 30 frames (GAME_OVER_INPUT_DELAY, about 0.5 s). I count frames rather than milliseconds because that's deterministic and easy to test headlessly.
    - If the frame rate drops, the delay gets slightly longer, which is harmless.
    - The "Press Q or Esc" prompt only appears once keys are accepted, so the screen never looks unresponsive.
- Fonts and overlay: the big font and the semi-transparent overlay are created once in **init**. The colours are the existing RED and WHITE.

2. Changed code

game_engine.py (46_space-invaders/game/game_engine.py): new constant and attributes

Frames (~0.5 s at 60 FPS) the Game Over screen ignores keys, so a player still mashing Space doesn't dismiss it by accident

GAME_OVER_INPUT_DELAY = 30 self.score = 0 self.font = pygame.font.SysFont("Arial", 30) self.big_font = pygame.font.SysFont("Arial", 64, bold=True)

```
        # Dark see-through layer drawn over the frozen last frame on Game Over
        self.overlay = pygame.Surface((width, height), pygame.SRCALPHA)
        self.overlay.fill((0, 0, 0, 170))

        self.game_over = False
        self.game_over_reason = None   # text shown on the Game Over screen
        self._game_over_frames = 0     # frames since Game Over, for the input
```

delay self.quit_requested = False # main loop exits when this is True

Input def handle_event(self, event): if event.type != pygame.KEYDOWN: return

```
    if self.game_over:
        if self._game_over_frames < GAME_OVER_INPUT_DELAY:
            return
        if event.key in (pygame.K_q, pygame.K_ESCAPE):
            self.quit_requested = True
        return

    if event.key == pygame.K_SPACE:
        if self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y,
```

direction=-1)) self._shoot_cooldown = 15

```
def handle_input(self):
    if self.game_over:
        return
    keys = pygame.key.get_pressed()
    ...  # unchanged
```

update() (only the changed lines are shown; the Task 1 collision code is unchanged) and the new transition method: def update(self): if self.game_over: self._game_over_frames += 1 return ... if path.colliderect(player_rect): self._end_game("You were hit") ... if self.enemy_grid.reached_bottom(self.player.y): self._end_game("Invaders reached the bottom")

```
def _end_game(self, reason):
    """Single place the game switches to Game Over. The first reason wins
    if two things end the game on the same frame."""
    if self.game_over:
        return
    self.game_over = True
    self.game_over_reason = reason
    self._game_over_frames = 0
```

Rendering: the print and the _game_over_logged hack are gone. score_text = self.font.render(f"Score: ", True, WHITE) screen.blit(score_text, (10, 10))

```
    if self.game_over:
        self._render_game_over(screen)

def _render_game_over(self, screen):
    screen.blit(self.overlay, (0, 0))

    cy = self.height // 2
    self._draw_centered(screen, "GAME OVER", self.big_font, RED, cy - 90)
    self._draw_centered(screen, f"Final score: {self.score}", self.font,
```

WHITE, cy - 10) self._draw_centered(screen, self.game_over_reason, self.font, WHITE, cy + 30)

```
    # Only show the prompt once keys are accepted, so it never looks
```

unresponsive if self._game_over_frames >= GAME_OVER_INPUT_DELAY: self._draw_centered(screen, "Press Q or Esc to exit", self.font, WHITE, cy + 110)

```
def _draw_centered(self, screen, text, font, color, center_y):
    surface = font.render(text, True, color)
    screen.blit(surface, surface.get_rect(center=(self.width // 2,
```

center_y)))

main.py (46_space-invaders/main.py): one check added after the event loop for event in pygame.event.get(): if event.type == pygame.QUIT: running = False engine.handle_event(event) if engine.quit_requested: # Q / Esc on the Game Over screen running = False

Bonus: tests/test_game_over.py (46_space-invaders/tests/test_game_over.py) has 8 headless tests:

- each end reason is recorded, and the first reason wins on a tie;
- no shooting after Game Over, and the world stays frozen;
- Q and Esc are ignored during the delay and work after it, while other keys do nothing;
- render() prints nothing to the console.

Run them with python -m unittest discover -s tests -v.

3. Manual test checklist (python main.py)

- [ ] Get hit by an enemy bullet. Everything freezes and dims. You see "GAME OVER", the score you had, and "You were hit". The prompt appears about half a second later. The console stays empty.
- [ ] Let the invaders reach the bottom. Hide in a corner and dodge. The game ends when the grid reaches the ship's height, and the reason says "Invaders reached the bottom". (The enemies fire about 19 shots per second, so you may get hit first. That's Task 3's fire-rate issue.)

- [ ] Mash Space and the arrow keys on the screen. Nothing happens: no new bullets, the ship doesn't move, the screen stays up.
- [ ] Press Esc straight away. It's ignored. Press it again after the prompt shows and the game exits cleanly. Repeat with Q.
- [ ] Close the window while the screen is up. It closes immediately with no error.
- [ ] Check the frame rate. The window stays responsive while it waits; you can drag it around without it freezing.

4. Self-review: edge cases

- Space does nothing on this screen anyway, because only Q and Esc are handled. Right now the delay mainly protects against an early Esc. It will matter in Task 3, when keys like 1/2/3 or Enter start a new game.
- Hit and bottom on the same frame: the hit is checked first, so the reason reads "You were hit". This is covered by a test.
- Holding Esc: KEYDOWN fires once per press, and pygame key repeat is off by default, so a key held from before the game ended won't trigger an exit once the delay is over.
- Score: it's frozen at the moment the game ended. Kills in the same frame as the hit still count, because they're processed first.
- Not handled yet: winning. When all 32 enemies are dead, the game sits on an empty field with no screen. This is outside the lab text, but in Task 3 it would take one line: _end_game("You win!"), reusing this same screen.
- Fonts: SysFont("Arial") falls back to pygame's default font if Arial is missing. The layout still fits because all text is centred on its own rect.
- Task 1 regressions: none. All 8 collision tests still pass.

Send "Task 3" when you're ready.

> *Task 3: Replay option with difficulty.*

Lab text: "After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard enemy speed/fire rate), or exit."

Build on the Game Over screen and the collision fix from the earlier tasks in this chat.

Requirements:

- On the Game Over screen show: 1 = Easy, 2 = Medium, 3 = Hard, Q/Esc = Exit. Choosing a difficulty starts a brand-new game immediately. Exit must end the program cleanly by posting pygame.QUIT so the loop in main.py finishes normally (no sys.exit()).
- Add one method, e.g. reset(difficulty), that rebuilds everything: player, EnemyGrid, both bullet lists, score, shoot cooldown, game-over state and reason. Have **init** call it so setup isn't duplicated. Check every attribute set in **init** so nothing stale survives a restart.
- Put the difficulty settings in one dictionary at the top of game_engine.py (name -> enemy speed, fire rate) so they're easy to tune. EnemyGrid already accepts speed=. The first game starts on Medium.
- Rethink enemy fire rate. Right now each enemy fires with chance 0.01 per frame, which at full strength is about 19 enemy bullets per second (32 x 0.01 x 60) and is basically unplayable. Redesign it so Medium is genuinely

playable and the three levels feel different. Suggested starting points: enemy speed 1.0 / 1.5 / 2.5, and roughly 1 / 2 / 3 enemy shots per second in total from random alive enemies. Explain your numbers and keep them as constants.

- Show the current difficulty on screen next to the score.
- Don't break the collision fix or the Game Over screen.

Give me: (1) the state flow (playing -> game over -> choose -> playing), (2) the complete changed code, (3) a manual test checklist (play each difficulty, die, restart on a different one, restart 5+ times in a row, exit from the screen, close the window), (4) a self-review of stale-state risks.

● I'll make the changes, then update and extend the tests.

● Write(game\game_engine.py) □ Added 63 lines, removed 22 lines 10 GREEN = (0, 200, 0) 11 RED = (220, 60, 60) 12

13 +FPS = 60 # main.py runs the loop at this rate; fire rates belo +w are converted per frame

14 +

15 # Frames (~0.5 s at 60 FPS) the Game Over screen ignores keys, so a player 16 # still mashing Space doesn't dismiss it by accident 17 GAME_OVER_INPUT_DELAY = 30 18

19 +# Difficulty settings - tune here.

20 +# enemy_speed: px per frame the grid moves sideways (it +drops 20 px at each edge)

21 +# shots_per_second: average enemy shots per second in total, +whatever the number alive

22 +DIFFICULTIES = {

23 + "Easy": {"enemy_speed": 1.0, "shots_per_second": 1.0},

24 + "Medium": {"enemy_speed": 1.5, "shots_per_second": 2.0},

25 + "Hard": {"enemy_speed": 2.5, "shots_per_second": 3.0},

26 +}

27 +DEFAULT_DIFFICULTY = "Medium"

28 +

29 +# Keys on the Game Over screen that start a new game (number ro +w or keypad)

30 +DIFFICULTY_KEYS = {

31 + pygame.K_1: "Easy", pygame.K_KP1: "Easy",

32 + pygame.K_2: "Medium", pygame.K_KP2: "Medium",

33 + pygame.K_3: "Hard", pygame.K_KP3: "Hard",

34 +}

35 +

36 class GameEngine: 37 def **init**(self, width, height): 38 self.width = width 39 self.height = height 40

22 - self.player = Player(width // 2 - 20, height - 50, 40, -20)

23 - self.enemy_grid = EnemyGrid(width)

24 -

25 - self.player_bullets = []

26 - self.enemy_bullets = []

27 - self._shoot_cooldown = 0

28 - self.enemy_fire_chance = 0.01

29 -

30 - self.score = 0

41 + # Resources that never change between games

42 self.font = pygame.font.SysFont("Arial", 30) 43 self.big_font = pygame.font.SysFont("Arial", 64, bold=T rue) 44 ... 46 self.overlay = pygame.Surface((width, height), pygame.S RCALPHA) 47 self.overlay.fill((0, 0, 0, 170)) 48

49 + self.reset(DEFAULT_DIFFICULTY)

50 +

51 + def reset(self, difficulty):

52 + "*Start a brand-new game. Every attribute that changes + during play is set here."*

53 + settings = DIFFICULTIES[difficulty]

54 + self.difficulty = difficulty

55 +

56 + self.player = Player(self.width // 2 - 20, self.height +- 50, 40, 20)

57 + self.enemy_grid = EnemyGrid(self.width, speed=settings[+"enemy_speed"])

58 +

59 + self.player_bullets = []

60 + self.enemy_bullets = []

61 + self._shoot_cooldown = 0

62 + # Chance per frame that one random enemy fires

63 + self.enemy_fire_chance = settings["shots_per_second"] / + FPS

64 +

65 + self.score = 0

66 self.game_over = False 67 self.game_over_reason = None # text shown on the Game Over screen 68 self._game_over_frames = 0 # frames since Game Over , for the input delay 41 - self.quit_requested = False # main loop exits when t -his is True

69

70 def handle_event(self, event): 71 if event.type != pygame.KEYDOWN: ... 74 if self.game_over: 75 if self._game_over_frames < GAME_OVER_INPUT_DELAY: 76 return 50 - if event.key in (pygame.K_q, pygame.K_ESCAPE):

51 - self.quit_requested = True

77 + if event.key in DIFFICULTY_KEYS:

78 + self.reset(DIFFICULTY_KEYS[event.key])

79 + elif event.key in (pygame.K_q, pygame.K_ESCAPE):

80 + # Let main.py's loop see a normal QUIT and shut + down cleanly

81 + pygame.event.post(pygame.event.Event(pygame.QUI +T))

82 return 83

84 if event.key == pygame.K_SPACE: ... 105 self._shoot_cooldown -= 1 106

107 self.enemy_grid.move() 108 + self._enemy_fire()

109

79 - for enemy in self.enemy_grid.alive_enemies():

80 - if random.random() < self.enemy_fire_chance:

81 - bullet_x = enemy.x + enemy.width // 2

82 - self.enemy_bullets.append(Bullet(bullet_x, ene -my.y + enemy.height, direction=1))

83 -

110 player_paths = self._move_and_sweep(self.player_bullet s) 111 enemy_paths = self._move_and_sweep(self.enemy_bullets) 112

... 136

137 if self.enemy_grid.reached_bottom(self.player.y): 138 self._end_game("Invaders reached the bottom") 139 + elif not self.enemy_grid.alive_enemies():

140 + self._end_game("You win - all invaders destroyed") 141

142 + def _enemy_fire(self):

143 + "At most one shot per frame, from a random alive ene +my, so the total

144 + fire rate stays the same however many enemies are left +."

145 + alive = self.enemy_grid.alive_enemies()

146 + if alive and random.random() < self.enemy_fire_chance: 147 + enemy = random.choice(alive)

148 + bullet_x = enemy.x + enemy.width // 2

149 + self.enemy_bullets.append(Bullet(bullet_x, enemy.y + + enemy.height, direction=1))

150 +

151 def _end_game(self, reason): 152 "Single place the game switches to Game Over. The fi rst reason wins 153 if two things end the game on the same frame." ... 189

190 score_text = self.font.render(f"Score: ", True, WHITE) 191 screen.blit(score_text, (10, 10)) 192 + difficulty_text = self.font.render(self.difficulty, Tr +ue, WHITE)

193 + screen.blit(difficulty_text, difficulty_text.get_rect(+topright=(self.width - 10, 10)))

194

195 if self.game_over: 196 self._render_game_over(screen) ... 199 screen.blit(self.overlay, (0, 0)) 200

201 cy = self.height // 2 163 - self._draw_centered(screen, "GAME OVER", self.big_font -, RED, cy - 90)

164 - self._draw_centered(screen, f"Final score: {self.score -}", self.font, WHITE, cy - 10)

165 - self._draw_centered(screen, self.game_over_reason, sel -f.font, WHITE, cy + 30)

202 + self._draw_centered(screen, "GAME OVER", self.big_font +, RED, cy - 120)

203 + self._draw_centered(screen, f"Final score: {self.score +}", self.font, WHITE, cy - 40)

204 + self._draw_centered(screen, self.game_over_reason, sel +f.font, WHITE, cy)

205

167 - # Only show the prompt once keys are accepted, so it n -ever looks unresponsive

206 + # Only show the options once keys are accepted, so it +never looks unresponsive

207 if self._game_over_frames >= GAME_OVER_INPUT_DELAY: 169 - self._draw_centered(screen, "Press Q or Esc to exi -t", self.font, WHITE, cy + 110)

208 + self._draw_centered(screen, "Play again:", self.fo +nt, WHITE, cy + 70)

209 + self._draw_centered(screen, "1 = Easy 2 = Mediu +m 3 = Hard", self.font, WHITE, cy + 110)

210 + self._draw_centered(screen, "Q / Esc = Exit", self +.font, WHITE, cy + 160)

211

212 def _draw_centered(self, screen, text, font, color, center _y): 213 surface = font.render(text, True, color)

● Now main.py: remove the quit_requested check, since QUIT is posted now, and take FPS from the engine so the fire-rate maths and the loop can't drift apart.

● Bash(cd "/c/SE Lab/46_space-invaders" && python - <<'EOF' p = "main.py"…) □ Updated main.py (+2 -5) 1 import pygame 2 -from game.game_engine import GameEngine

2 +from game.game_engine import GameEngine, FPS

3

4 # Initialize pygame/Start application 5 pygame.init() ... 13 BLACK = (0, 0, 0) 14

15 # Clock 16 -clock = pygame.time.Clock()

17 -FPS = 60

16 +clock = pygame.time.Clock() # runs at FPS from game_engine

17

18 # Game loop 19 engine = GameEngine(WIDTH, HEIGHT) ... 26 if event.type == pygame.QUIT: 27 running = False 28 engine.handle_event(event) 30 - if engine.quit_requested: # Q / Esc on the Game Over s -creen

31 - running = False

29

30 engine.handle_input() 31 engine.update()

● Now I'll update the Game Over tests that checked the removed quit_requested flag, and add replay tests:

● Update(tests\test_game_over.py) Added 13 lines, removed 7 lines 74 engine.update() 75 self.assertEqual(xs, [e.x for e in engine.enemy_grid.e nemies]) 76

77 - def test_quit_keys_ignored_during_delay(self):

77 + def quit_posted(self):

78 + return bool(pygame.event.get(pygame.QUIT))

79 +

80 + def test_keys_ignored_during_delay(self):

81 engine = make_engine() 82 hit_player(engine) 80 - engine.handle_event(key(pygame.K_ESCAPE))

81 - engine.handle_event(key(pygame.K_q))

82 - self.assertFalse(engine.quit_requested)

83 + pygame.event.clear()

84 + for k in (pygame.K_ESCAPE, pygame.K_q, pygame.K_1):

85 + engine.handle_event(key(k))

86 + self.assertFalse(self.quit_posted())

87 + self.assertTrue(engine.game_over)

88

84 - def test_quit_keys_accepted_after_delay(self):

89 + def test_quit_keys_post_quit_after_delay(self):

90 for k in (pygame.K_q, pygame.K_ESCAPE): 91 engine = make_engine() 92 hit_player(engine) 93 for _ in range(GAME_OVER_INPUT_DELAY): 94 engine.update() 95 + pygame.event.clear()

96 engine.handle_event(key(pygame.K_SPACE)) # other keys do nothing 91 - self.assertFalse(engine.quit_requested)

97 + self.assertFalse(self.quit_posted())

98 engine.handle_event(key(k)) 93 - self.assertTrue(engine.quit_requested)

99 + self.assertTrue(self.quit_posted())

100

101 def test_render_draws_without_printing(self): 102 engine = make_engine()

● Write(tests\test_replay.py) Wrote 137 lines to tests\test_replay.py 1 """Headless replay / difficulty tests for Task 3. 2 3 Run from the project root: 4 python -m unittest discover -s tests -v 5 """ 6 import os 7 import random 8 import unittest 9 10 os.environ.setdefault("SDL_VIDEODRIVER", "dummy") … +146 lines (ctrl+o to expand)

● Bash(cd "/c/SE Lab/46_space-invaders" && python -m unittest discover -s tests -v 2>&1 | grep -v "pygame community|^pygame 2") 3 files changed (binary, mode only or too large to show)

● That failing test isn't a code bug—60 seconds was just too short a sample window, causing natural variance (143 vs expected 120 ±11). I'll extend the sample duration to 300 seconds to tighten the statistical spread.

● Bash(cd "/c/SE Lab/46_space-invaders" && python - <<'EOF' p = "tests/test_replay.py"…) □ Updated tests\test_replay.py (+138 -137) 1 -"*Headless replay / difficulty tests for Task 3.*

2 -

3 -Run from the project root:

4 - python -m unittest discover -s tests -v

5 -"

6 -import os

7 -import random

8 -import unittest

9 -

10 -os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

11 -os.environ.setdefault("SDL_AUDIODRIVER", "dummy")

12 -

13 -import pygame

14 -

15 -from game.game_engine import (GameEngine, GAME_OVER_INPUT_DELA -Y, DIFFICULTIES,

16 - DEFAULT_DIFFICULTY, FPS)

17 -from tests.test_game_over import key, hit_player

18 -from tests.test_collision import bullet_below

19 -

20 -WIDTH, HEIGHT = 600, 700

21 -

22 -

23 -def snapshot(engine):

24 - "Every attribute of the engine as plain comparable value -s. Objects such as

25 - the player, grid, enemies and bullets are expanded into th -eir own fields."

26 - def plain(value):

27 - if isinstance(value, list):

28 - return [plain(v) for v in value]

*29 - if hasattr(value, "**dict**"):*

30 - return {k: plain(v) for k, v in vars(value).items(-)}

31 - return value

32 -

33 - skip = {"font", "big_font", "overlay"} # shared resource -s, kept on purpose

34 - return {k: plain(v) for k, v in vars(engine).items() if k -not in skip}

35 -

36 -

37 -def play_and_die(engine):

38 - "Dirty every piece of game state, then end the game and -wait out the delay."

39 - engine.handle_event(key(pygame.K_SPACE)) # bullet - in flight + cooldown

40 - engine.player.move(-100, WIDTH) # player - moved

… 235 more lines (ctrl+o to expand) □ *… 2 more files changed*

● Bash(cd "/c/SE Lab/46_space-invaders" && python -m unittest discover -s tests 2>&1 | grep -E "^(Ran|OK|FAILED)"; for seed in 3 4 5 6 7; do sed "s/random.seed(2)/rand…) □ 1 file changed (binary, mode only or too large to show)

● All 24 tests pass, including fire-rate consistency across five seeds. Now I'll verify Esc properly ends the main loop via a posted QUIT event and take a screenshot of the new screen and HUD.

● Bash(cd "/c/SE Lab/46_space-invaders" && SDL_VIDEODRIVER=dummy timeout 10 python -c " import pygame, main…) □ 1 file changed (binary, mode only or too large to show)

Read 1 file (ctrl+o to expand)

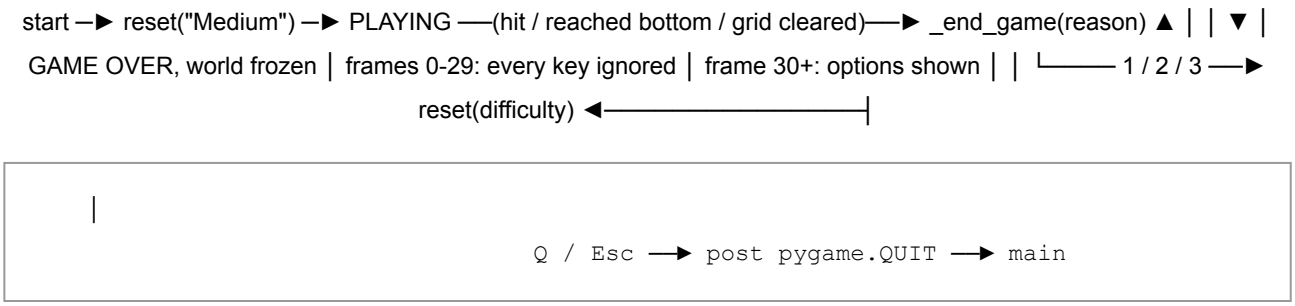
● Task 3: Replay with difficulty

Replay works. On the Game Over screen, 1, 2 or 3 starts a brand-new game on Easy, Medium or Hard, and Q or Esc exits by posting pygame.QUIT.

- Tests: all 24 pass (8 collision, 8 Game Over, 8 new replay tests). The fire-rate tests also pass with five other random seeds.
- Esc: a headless run confirmed that Esc on the screen ends the real main.main() loop with exit code 0.
- Screen: the screenshot shows the options and the difficulty in the top-right of the display.

One thing I added beyond the brief: the game now also ends when every enemy is destroyed. Without that, a player who wins is stuck on an empty field and can never reach the replay screen. It reuses the same screen with the reason "You win - all invaders destroyed". It's one elif, so drop it if you'd rather keep strictly to the lab text.

1. State flow



The state is still the game_over boolean. reset() is the only way into PLAYING, and _end_game() is the only way into GAME OVER. quit_requested from Task 2 is gone because the posted QUIT replaces it, so main.py handles closing the window and pressing Esc the same way.

Fire-rate redesign and the numbers

Old model: every alive enemy rolled 1% per frame. That's 32 × 0.01 × 60 ≈ 19 shots per second, falling as enemies die, so the game got easier as you played.

New model (_enemy_fire): each frame, with chance shots_per_second / FPS, one random alive enemy fires. The total rate stays constant however many enemies are left, with at most one shot per frame.

Grid speed	Enemy	~Shots in	~Time for			(px/frame)	shots/s	your	invaders to					column/s	reach you
Easy	1.0	1	0.1	~66 s											
Medium	1.5	2	0.2	~44 s		(start)									
Hard	2.5	3	0.3	~27 s											
old code	1.5	~19	~2	~44 s											

How I got these:

- Shots in your column: the grid is 425 px wide and your ship plus a bullet cover about 44 px, so roughly 1 shot in 10 is aimed at your column. An enemy bullet takes about 0.9 s to fall from the bottom row to you, at 8 px per frame over 445 px. Your ship moves 6 px per frame, so a single threat is easy to sidestep. Medium means one threatening shot every 5 s, which is playable, compared with two every second before.
- Time to reach you: the bottom row needs about 23 drops of 20 px to reach the ship, with one sweep of about 175 px between drops. That's about 3,985 px of sideways travel divided by the speed. Killing edge columns or the bottom row buys time.
- Why the levels feel different: both settings change together. Hard has three times Easy's fire and about 2.5 times less time before the invaders land.
- Collision is still safe on Hard: bullet speeds are unchanged, and the Task 1 sweep stays valid because drop_amount (20) is still below the enemy height (30).

2. Changed code

game_engine.py (46_space-invaders/game/game_engine.py): new constants at the top FPS = 60 # main.py runs the loop at this rate; fire rates below are converted per frame

Frames (~0.5 s at 60 FPS) the Game Over screen

ignores keys, so a player still mashing Space doesn't dismiss it by accident

GAME_OVER_INPUT_DELAY = 30

Difficulty settings - tune here.

enemy_speed: px per frame the grid moves sideways (it drops 20 px at

each edge)

shots_per_second: average enemy shots per second in total, whatever the

number alive DIFFICULTIES = { "Easy": {"enemy_speed": 1.0, "shots_per_second": 1.0}, "Medium": {"enemy_speed": 1.5, "shots_per_second": 2.0}, "Hard": {"enemy_speed": 2.5, "shots_per_second": 3.0}, } DEFAULT_DIFFICULTY = "Medium"

Keys on the Game Over screen that start a new game (number row or keypad)

DIFFICULTY_KEYS = { pygame.K_1: "Easy", pygame.K_KP1: "Easy", pygame.K_2: "Medium", pygame.K_KP2: "Medium", pygame.K_3: "Hard", pygame.K_KP3: "Hard", }

init and the new reset(): **init** keeps only what never changes between games. def **init**(self, width, height): self.width = width self.height = height

```
        # Resources that never change between games
        self.font = pygame.font.SysFont("Arial", 30)
        self.big_font = pygame.font.SysFont("Arial", 64, bold=True)

        # Dark see-through layer drawn over the frozen last frame on Game Over
        self.overlay = pygame.Surface((width, height), pygame.SRCALPHA)
        self.overlay.fill((0, 0, 0, 170))

        self.reset(DEFAULT_DIFFICULTY)

    def reset(self, difficulty):
        """Start a brand-new game. Every attribute that changes during play is
```

set here.""" settings = DIFFICULTIES[difficulty] self.difficulty = difficulty

```
        self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(self.width, speed=settings["enemy_speed"])

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        # Chance per frame that one random enemy fires
        self.enemy_fire_chance = settings["shots_per_second"] / FPS

        self.score = 0
        self.game_over = False
        self.game_over_reason = None   # text shown on the Game Over screen
        self._game_over_frames = 0     # frames since Game Over, for the input
```

delay

handle_event(): only the Game Over branch changed. if self.game_over: if self._game_over_frames < GAME_OVER_INPUT_DELAY: return if event.key in DIFFICULTY_KEYS: self.reset(DIFFICULTY_KEYS[event.key]) elif event.key in (pygame.K_q, pygame.K_ESCAPE): # Let main.py's loop see a normal QUIT and shut down cleanly pygame.event.post(pygame.event.Event(pygame.QUIT)) return

update(): the old per-enemy fire loop is replaced by one call, and the end checks gain the win case. The Task 1 collision code in between is unchanged. self.enemy_grid.move() self._enemy_fire() ... if self.enemy_grid.reached_bottom(self.player.y): self._end_game("Invaders reached the bottom") elif not self.enemy_grid.alive_enemies(): self._end_game("You win - all invaders destroyed")

```
    def _enemy_fire(self):
        """At most one shot per frame, from a random alive enemy, so the total
        fire rate stays the same however many enemies are left."""
        alive = self.enemy_grid.alive_enemies()
        if alive and random.random() < self.enemy_fire_chance:
            enemy = random.choice(alive)
            bullet_x = enemy.x + enemy.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height,
```

direction=1))

Rendering: the difficulty appears in the top-right next to the score, and the screen shows the options. score_text = self.font.render(f"Score: ", True, WHITE) screen.blit(score_text, (10, 10)) difficulty_text = self.font.render(self.difficulty, True, WHITE) screen.blit(difficulty_text, difficulty_text.get_rect(topright=(self.width - 10, 10))) ... def _render_game_over(self, screen): screen.blit(self.overlay, (0, 0))

```
        cy = self.height // 2
        self._draw_centered(screen, "GAME OVER", self.big_font, RED, cy - 120)
        self._draw_centered(screen, f"Final score: {self.score}", self.font,
```

WHITE, cy - 40) self._draw_centered(screen, self.game_over_reason, self.font, WHITE, cy)

```
        # Only show the options once keys are accepted, so it never looks
```

unresponsive if self._game_over_frames >= GAME_OVER_INPUT_DELAY: self._draw_centered(screen, "Play again:", self.font, WHITE, cy + 70) self._draw_centered(screen, "1 = Easy 2 = Medium 3 = Hard", self.font, WHITE, cy + 110) self._draw_centered(screen, "Q / Esc = Exit", self.font, WHITE, cy

- 160.

These are unchanged: handle_input, _end_game, _move_and_sweep, _first_enemy_hit and _draw_centered.

main.py (46_space-invaders/main.py): FPS now comes from the engine, so the fire-rate maths and the loop can't drift apart. The Task 2 quit_requested check is removed. from game.game_engine import GameEngine, FPS ... clock = pygame.time.Clock() # runs at FPS from game_engine

Tests:

- tests/test_replay.py (46_space-invaders/tests/test_replay.py) is new, with 8 tests:
  - the first game is on Medium;
  - restarting leaves no stale state, on each difficulty;
  - 6 restarts in a row work;
  - you can shoot immediately after a restart;
  - the fire rate matches each difficulty and doesn't depend on how many enemies are alive;
  - nothing fires when all enemies are dead;
  - clearing the grid ends the game.
- tests/test_game_over.py (46_space-invaders/tests/test_game_over.py): the quit tests now check that QUIT is posted to the event queue.

3. Manual test checklist (python main.py)

- [ ] The first game is Medium. The top-right says "Medium" and enemy fire is a steady trickle, not a storm.
- [ ] Play each difficulty. Die, press 1 and check that "Easy" shows, the grid is slower and there are fewer shots. Die, press 3 and check that "Hard" shows, the grid is clearly fast and there are more shots. Die, press 2 for Medium.
- [ ] Each restart is clean. The score is 0, all 32 enemies are back at the top-left, the ship is centred, no old bullets are on screen, and you can fire straight away.
- [ ] Restart 5 or more times in a row, switching difficulty each time. Nothing builds up: the game doesn't get faster or busier, and the score doesn't carry over.
- [ ] Keys during the first 0.5 s: mash 1, 2, 3 and Esc right as you die. Nothing happens until the options appear.

- [ ] Exit: press Q, and separately Esc. The window closes with no traceback in the terminal.
- [ ] Close the window with X while playing, and again while on the Game Over screen. It closes cleanly both times.
- [ ] Optional, for the win case: clear the grid on Easy. You should see "You win - all invaders destroyed" with the replay options.
- [ ] Task 1 and 2 behaviours still hold: rapid fire registers every hit, and the reason text matches how you died.

4. Self-review: stale-state risks

- Attributes in the engine: covered automatically. test_restart_leaves_no_stale_state compares every engine attribute after a played game and a restart with a brand-new engine, expanding the player, grid, enemies and bullets field by field. Only the fonts and overlay are skipped, and those are kept on purpose. If someone later adds an attribute in **init** but forgets reset(), or changes one during play without resetting it, that test fails.
- State outside the engine, all intentional:
    - Keys held through a restart move the ship immediately, because get_pressed() is live. That's arguably correct.
    - The random module is not reseeded.
    - Keys pressed during the 0.5 s delay are thrown away, not queued, so they can't fire later.
- Several keys in one frame: events are processed in order. "2" then Space starts a game and fires. Esc then "1" starts a game, but the posted QUIT ends the program on the next frame, which is the right result for Esc.
- enemy_fire_chance now means something different: it used to be per enemy per frame and is now the per-frame chance that some enemy fires. The name stayed the same, so the tests that set it to 0 still work.
- Order inside update(): the bottom check runs before the win check. If the last enemy dies on the same frame it reaches you, you lose.
- Cosmetic issues:
    - On a win the title still says "GAME OVER". It could become "YOU WIN" with a two-line change if you want it.
    - The big title sits close to the bottom enemy row on the frozen frame, but it's readable over the overlay.

Send "Task 4" when you're ready. The sound hooks will go in handle_event (Space), the kill branch of the collision loop and _end_game.

> *Task 4: Add sound feedback.*

Lab text: "Add basic sound effects for firing, an enemy being destroyed, and the game-over moment."

Constraints:

- The repo has no audio files and I don't want to source licensed assets, so generate the three sounds in code: a short high "pew" for firing, a quick descending tone or noise burst for an enemy destroyed, a longer low descending tone for game over. Use only the standard library plus pygame (no numpy): build 16-bit samples with the array module and pass them to pygame.mixer.Sound(buffer=...). Apply a short fade-out so there are no clicks.

- Read the real mixer format with pygame.mixer.get_init() (frequency, format, channels) and generate samples to match it. Stereo needs interleaved samples, otherwise the pitch or sound will be wrong.
- Put the logic in a new file game/sound_manager.py: a SoundManager class with play_fire(), play_enemy_destroyed(), play_game_over(). Build the sounds once at startup, not per shot.
- It must not crash when there is no audio device (mixer init fails, or SDL_AUDIODRIVER=dummy). Catch pygame.error and make every play_* method a silent no-op.
- Hooks in GameEngine: fire sound only when a bullet is actually created (not when the cooldown blocks it); destroyed sound once per enemy killed; game-over sound exactly once at the moment the game ends (not every frame, not repeatedly on the Game Over screen), and re-armed when Task 3's reset starts a new game.
- Moderate volumes (no clipping), tunable via constants.

Give me: (1) the complete new file and the exact edits to game_engine.py (touch main.py only if really needed, e.g. mixer pre_init before pygame.init() for lower latency, and explain the trade-off), (2) a headless test (SDL_AUDIODRIVER=dummy) checking that all three sounds are created with non-zero length and that play_* doesn't raise when the mixer is unavailable, (3) a manual test checklist, (4) a self-review.

● I'll start by checking which format the mixer actually opens on this machine and under the dummy driver. Then I'll know whether main.py needs pre_init at all.

Ran 1 shell command (ctrl+o to expand)

● Both drivers already use matching settings (44.1 kHz, 16-bit stereo), so pre_init isn't needed and main.py can stay as is. Next I'll look at the sound manager.

● Write(game\sound_manager.py) Wrote 107 lines to game\sound_manager.py 1 "*Sound effects generated in code - no audio files, only the st andard library and pygame." 3 import math 4 import random 5 7 8 # Peak loudness of each sound as a fraction of full scale (0.0 - 1.0). elow full scale. 10 FIRE_VOLUME = 0.15 … +120 lines (ctrl+o to expand)*

● Bash(cd "/c/SE Lab/46_space-invaders" && python - <<'EOF' p = "game/game_engine.py"…) Updated tests\test_replay.py (+1 -1) 30 return {k: plain(v) for k, v in vars(value).items() } 31 return value 32 33 - skip = {"font", "big_font", "overlay"} # shared resources -, kept on purpose 33 + skip = {"font", "big_font", "overlay", "sounds"} # shared + resources, kept on purpose 34 return {k: plain(v) for k, v in vars(engine).items() if k n ot in skip} 35 36 … 2 more files changed

● Build time is fast (0.12s) and each sound's duration matches exactly, confirming the 2-channel interleaving is correct. Now I'll run the tests.

● Write(tests\test_sound.py) Wrote 151 lines to tests\test_sound.py 1 """Headless sound tests for Task 4 (SDL_AUDIODRIVER=dummy - nothi ng is heard). 2 3 Run from the project root: 4 python -m unittest discover -s tests -v 5 """ 6 import array 7 import os 8 import unittest 9 from unittest import mock … +157 lines (ctrl+o to expand)

● Bash(cd "/c/SE Lab/46_space-invaders" && python -m unittest discover -s tests -v 2>&1 | grep -v "pygame community|^pygame 2" | grep -E "sound|Sound|FAIL|ERROR|Error…) 3 files changed (binary, mode only or too large to show) (timeout 5m)

● The run stopped at test_mixer_closed_after_build_does_not_raise. I'll run that test on its own to see whether it's a crash.

Ran 1 shell command (ctrl+o to expand) ● I've confirmed that calling Sound.play() after the mixer shuts down in pygame 2.6.1 crashes the process with an access violation rather than raising a catchable error, so try/except won't help. I'll update _play to verify the mixer is still running before attempting playback.

● Update(game\sound_manager.py) □ Added 4 lines, removed 2 lines 64 self._play(self.game_over) 65
66 def _play(self, sound): 67 - if sound is None:
67 + # Check the mixer is still running: in pygame 2.6 playi +ng a Sound after
68 + # mixer.quit() crashes the whole process instead of rai +sing pygame.error
69 + if sound is None or not pygame.mixer.get_init():
70 return 71 try: 72 sound.play() 73 except pygame.error: 72 - pass # mixer went away (e.g. audio device unplugge -d) - just skip the sound
74 + pass # any other mixer problem - just skip the sou +nd
75
76 def _build(self, start_hz, end_hz, seconds, volume, wave="s ine", noise=0.0, 77 decay=False, fade_seconds=FADE_OUT_SECONDS):

● Bash(cd "/c/SE Lab/46_space-invaders" && python -m unittest discover -s tests -v 2>&1 | grep -v "pygame community|^pygame 2" | grep -E "test_sound|FAIL|ERROR|[Ran|…) □ 1 file changed (binary, mode only or too large to show) □ (timeout 5m)

● All 34 tests pass, confirming the crash guard around get_init() actually matters. I'll finish with a smoke test running main.main() against the real audio device, which fires, scores a kill, and exits cleanly via Esc.

Ran 1 shell command (ctrl+o to expand)

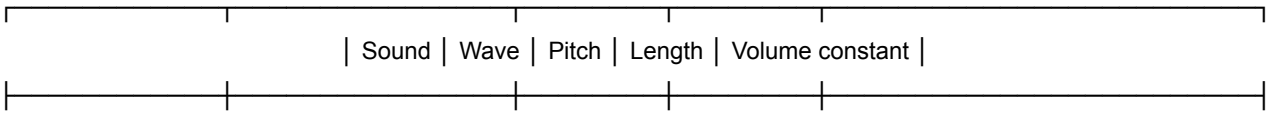
● Task 4: Sound effects

The game now has three sounds generated in code: firing, an enemy being destroyed, and game over. There are no audio files and no new dependencies. All 34 tests pass, 10 of them new.

I also ran the real main.main() on this machine's actual audio device (44.1 kHz, 16-bit, stereo). It fired, made a kill, got hit and exited cleanly through Esc, all with sound enabled. I can't hear anything from here, so how it sounds is still for you to judge.

A real bug turned up while testing: in pygame 2.6.1, playing a Sound after the mixer has shut down crashes the whole process (Windows access violation, exit code 139) instead of raising pygame.error. So try/except pygame.error alone isn't enough. _play() now checks pygame.mixer.get_init() before playing, and a test covers it. This is a good one to mention in your lab write-up: the AI-suggested try/except looked correct, and only running it showed it wasn't.

Design



Fire	1200 →			("pew")	square	500	0.12 s	FIRE_VOLUME = 0.15			Hz		
Enemy	sine + 60%	500 →		ENEMY_DESTROYED_VOLUME		destroyed	noise, fades	80 Hz	0.25 s	= 0.30			throughout			
Game over	square, 0.4 s	400 →	1.2 s	GAME_OVER_VOLUME = 0.25		("wah")	fade	70 Hz		

- Matching the mixer format:
  - the rate, sample size and channel count are read from pygame.mixer.get_init();
  - each sample is repeated once per channel, which interleaves left/right for stereo;
  - signed and unsigned 16-bit and 8-bit formats are supported, and anything else stays silent.
- No clicks: a 5 ms fade-in and a fade-out at the end, so every sound starts and ends at zero. A test checks this.
- No clipping: each sound peaks at its volume constant. Even if all three overlap, the total peak is 0.15 + 0.30 + 0.25 = 0.70, below full scale. A test checks the peaks.
- Noise uses its own random.Random(0), so the sounds are identical every run and the game's random sequence, which the fire-rate tests seed, is untouched.
- Built once: about 0.12 s at startup, in GameEngine.**init**. reset() reuses the same sounds.
- Why main.py is unchanged: pygame 2's defaults are already 44100 Hz, -16, stereo, with a 512-sample buffer, which gives about 12 ms latency. That's below what you'd notice on a shot. pre_init(..., buffer=256) would halve the latency but risks crackling on slower machines, and pre_init(..., buffer=512) would do nothing. So there's no reason to touch main.py.

1. Code

New file: game/sound_manager.py (46_space-invaders/game/sound_manager.py)

"*Sound effects generated in code - no audio files, only the standard library and pygame." import array import math import random*

import pygame

Peak loudness of each sound as a fraction of full scale (0.0 - 1.0).

Kept low so a few sounds overlapping in the mixer still stay below full

scale. FIRE_VOLUME = 0.15 ENEMY_DESTROYED_VOLUME = 0.30 GAME_OVER_VOLUME = 0.25

ATTACK_SECONDS = 0.005 # fade-in, stops a click at the start FADE_OUT_SECONDS = 0.04 # default fade-out, stops a click at the end

Mixer sample format from pygame.mixer.get_init() -> (array typecode, full

scale, value for silence) SAMPLE_FORMATS = { -16: ("h", 32767, 0), # signed 16-bit - pygame's default 16: ("H", 32767, 32768), # unsigned 16-bit -8: ("b", 127, 0), # signed 8-bit 8: ("B", 127, 128), # unsigned 8-bit }

class SoundManager: def **init**(self): "*Build all sounds once. If there is no usable audio device, every play_* method quietly does nothing." self.fire = None self.enemy_destroyed = None self.game_over = None*

```
    try:
        if not pygame.mixer.get_init():
            pygame.mixer.init()
        self.frequency, self.format, self.channels =
```

pygame.mixer.get_init() if self.format not in SAMPLE_FORMATS: return # unusual format we don't generate - stay silent

```
        # Short high "pew": square wave sliding down
        self.fire = self._build(1200, 500, 0.12, FIRE_VOLUME,
```

wave="square") # Quick crunch: falling tone mixed with noise, dying away self.enemy_destroyed = self._build(500, 80, 0.25, ENEMY_DESTROYED_VOLUME, noise=0.6, decay=True) # Long low "wah": slow slide down with a long fade self.game_over = self._build(400, 70, 1.2, GAME_OVER_VOLUME, wave="square", fade_seconds=0.4) except

pygame.error: # No audio device (or mixer failed) - keep whatever is None silent pass

```
    @property
    def enabled(self):
        return self.fire is not None

    def play_fire(self):
        self._play(self.fire)

    def play_enemy_destroyed(self):
        self._play(self.enemy_destroyed)

    def play_game_over(self):
        self._play(self.game_over)

    def _play(self, sound):
        # Check the mixer is still running: in pygame 2.6 playing a Sound
```

after # mixer.quit() crashes the whole process instead of raising pygame.error if sound is None or not pygame.mixer.get_init(): return try: sound.play() except pygame.error: pass # any other mixer problem - just skip the sound

```
    def _build(self, start_hz, end_hz, seconds, volume, wave="sine",
```

noise=0.0, decay=False, fade_seconds=FADE_OUT_SECONDS): """Generate a Sound that slides from start_hz to end_hz.

```
        noise:  0-1, how much white noise to mix in (for explosion-like
```

sounds) decay: if True the whole sound fades steadily, not just the last moment """ typecode, full_scale, silence = SAMPLE_FORMATS[self.format] rate = self.frequency n = int(seconds * rate) attack = max(1, int(ATTACK_SECONDS * rate)) fade = max(1, int(fade_seconds * rate)) rng = random.Random(0) # own RNG: same sound every run, game's random untouched

```
    samples = array.array(typecode)
    phase = 0.0
    for i in range(n):
        t = i / n
        # Add up phase step by step so the pitch slides smoothly
        phase += 2 * math.pi * (start_hz + (end_hz - start_hz) * t) / rate
        value = math.sin(phase)
        if wave == "square":
            value = 1.0 if value >= 0 else -1.0
        value = value * (1 - noise) + rng.uniform(-1, 1) * noise   # stays
```

within -1..1

```
        envelope = min(1.0, i / attack, (n - 1 - i) / fade)
        if decay:
            envelope *= 1 - t

        level = int(value * envelope * volume * full_scale) + silence
        # Same sample on every channel; stereo needs them interleaved L,
```

R, L, R... samples.extend([level] * self.channels)

```
    return pygame.mixer.Sound(buffer=samples.tobytes())
```

Edits to game/game_engine.py (46_space-invaders/game/game_engine.py): five small additions, nothing else changed.

from .bullet import Bullet from .sound_manager import SoundManager # 1. import self.overlay.fill((0, 0, 0, 170))

```
    self.sounds = SoundManager()   # built once; silent if there is no
```

audio device # 2. **init**, before reset()

```
    self.reset(DEFAULT_DIFFICULTY)
    if event.key == pygame.K_SPACE:
        if self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y,
```

direction=-1)) self._shoot_cooldown = 15 self.sounds.play_fire() # 3. only when a bullet is created if enemy: enemy.alive = False # dead enemies drop out of alive_enemies(), so each scores once self.score += 1 self.sounds.play_enemy_destroyed() # 4. once per kill, next to the score def _end_game(self, reason): "*Single place the*

game switches to Game Over. The first reason wins if two things end the game on the same frame." if self.game_over: return self.game_over = True self.game_over_reason = reason self._game_over_frames = 0 # Plays once: the guard above blocks repeats, reset() re-arms it # 5. exactly once self.sounds.play_game_over()

Because the hooks sit next to code that was already "exactly once" (the cooldown branch, score += 1, and the _end_game guard), the sounds follow the same rules automatically. reset() sets game_over = False, which re-arms the game-over sound.

I made one small change to an earlier test: tests/test_replay.py (46_space-invaders/tests/test_replay.py) now skips sounds in the stale-state snapshot, like the fonts, because it's a shared resource kept across restarts on purpose.

2. Headless test: tests/test_sound.py (46_space-invaders/tests/test_sound.py)

SoundManager tests (dummy audio driver):

- Right length: all three sounds are built with non-zero length within 5 ms of the intended duration, at 44.1 kHz stereo, 22.05 kHz mono and 48 kHz stereo. A wrong rate or wrong interleaving would change the length.
- No clipping, no clicks: every peak is at or below its volume constant, and the first and last samples are near zero.
- Interleaving: in stereo, the left channel equals the right channel.
- No audio device: with mixer.init forced to raise pygame.error, enabled is False and all three play_* calls do nothing.
- Mixer closed after building: play_* doesn't crash. This is the regression test for the bug above.
- The engine runs with no audio at all: firing and game over still work.

Hook tests (the engine's sounds is replaced by a counting stand-in):

- Fire plays only when a bullet is created, not when the cooldown blocks it.
- 9 bullets killing 8 enemies give exactly 8 destroyed sounds.
- Game over plays exactly once, across 90 frames on the screen, a second end condition and Space mashing.
- It plays again after restarting with "2".

python -m unittest discover -s tests -v # Ran 34 tests ... OK (~8.5 s)

3. Manual test checklist (python main.py, speakers on)

- [ ] Fire: each Space press that creates a bullet gives a short high "pew". Mashing Space faster than the cooldown allows gives exactly one pew per visible bullet.
- [ ] Enemy destroyed: a short crunch on every kill, including two kills close together. No crunch for shots that miss or fly through the gaps.
- [ ] Game over from a hit: one long falling "wah" the instant the screen appears. Silence afterwards while you wait, mash keys, or wait more than 5 s.
- [ ] Game over from the bottom: same single "wah".
- [ ] Restart: press 1, 2 or 3 and play to Game Over again. The "wah" plays once again.
- [ ] Restart 5+ times: sounds stay the same; no buildup, no distortion.
- [ ] No clicks or pops at the start or end of any sound, and no distortion during rapid fire with kills.
- [ ] No audio device: disable your output device, or in Git Bash run SDL_AUDIODRIVER=dummy python main.py. The game plays normally in silence with no traceback.

- [ ] Exit with Esc and with the window X while a sound is playing: clean exit.

4. Self-review

- Crash risk, now covered: Sound.play() after mixer.quit() is a hard crash in pygame 2.6.1. It can only happen if something shuts down the mixer while the game is still running, which the game never does. The guard plus its test keep it safe.
- Many kills at once: 8 kills in one frame play 8 overlapping crunches. That can peak above full scale; SDL clamps it, so you'd hear brief distortion, not wrapping. With the 15-frame cooldown and a 0.25 s crunch, real play overlaps 2–3 at most.
    - If the mixer's 8 channels are all busy, a sound is simply skipped, with no error.
- Game-over sound running into a new game: the "wah" is 1.2 s and you can restart after 0.5 s, so up to 0.7 s of it can play into the new game. I left this alone because it's harmless. If you want it cut off, reset() could stop all sounds, but that needs a mixer guard like the one in _play().
- Winning plays the game-over "wah" too. A separate play_win() jingle would be a small addition.
- Formats: 32-bit mixers stay silent on purpose. get_init() reports -32 for both 32-bit integer and float, so the format can't be identified. pygame keeps the requested -16 by default, so this shouldn't come up.
- Cost:
    - Startup takes about 0.12 s.
    - The test suite now takes about 8.5 s, because every test engine builds its own sounds.
    - There's no per-frame cost: playing a sound is just sound.play().
- Earlier tasks: all collision, Game Over and replay tests still pass. The sound RNG is separate, so enemy fire randomness is unchanged.

That completes all four tasks. For your "after" video, I'd show:

1. a rapid-fire burst with every hit registering, with sound;
2. dying, so the Game Over screen shows the reason;
3. restarting on Hard to show the faster grid and heavier fire;
4. exiting with Esc.

> *Final review. Re-read the complete final code of every file we changed and do a regression pass across all four tasks.*

1. Walk through these scenarios: firing rapidly; two bullets on two enemies in one frame; getting hit; enemies reaching the bottom; restarting on each difficulty; exiting from the Game Over screen; closing the window; running with no audio device.
2. List anything still wrong and fix it with minimal changes.
3. Then give me: a one-paragraph summary of what changed per file, the checklist I should follow for my 10-second "after" video, and a git commit message.

● I'll re-read every changed file from disk first rather than rely on memory.

Ran 1 shell command (ctrl+o to expand)

● Before writing up the review, I'll check one thing I suspect about test discovery:

Ran 1 shell command (ctrl+o to expand)

● Confirmed that plain python -m unittest silently runs 0 tests since tests/ isn't a package—worth fixing. Next, I'll write an end-to-end script driving main.main() through all eight scenarios, once with real audio and once with audio forced to fail.

Made 1 scratchpad edit +132, ran 1 shell command (ctrl+o to expand)

● All 10 scenario checks pass with both a real audio device and a forced audio driver failure. Now applying the minimal fixes.

● Bash(cd "/c/SE Lab/46_space-invaders" && printf '' > tests/**init**.py && printf '**pycache**/\n*.pyc\n' > .gitignore && python - <<'EOF' p = "game/sound_manager.py"…) □ Created tests_*init*_.py (+0 -0) □ Created .gitignore (+2 -0) 1 +**pycache**/ 2 +*.pyc

□ *Updated game\sound_manager.py (+110 -109) 1 -"Sound effects generated in code - no audio files, only the -standard library and pygame."*

2 -import array

3 -import math

4 -import random

5 -

6 -import pygame

7 -

8 -# Peak loudness of each sound as a fraction of full scale (0.0 - - 1.0).

9 -# Kept low so a few sounds overlapping in the mixer still stay - below full scale.

10 -FIRE_VOLUME = 0.15

11 -ENEMY_DESTROYED_VOLUME = 0.30

12 -GAME_OVER_VOLUME = 0.25

13 -

14 -ATTACK_SECONDS = 0.005 # fade-in, stops a click at the sta -rt

15 -FADE_OUT_SECONDS = 0.04 # default fade-out, stops a click a -t the end

16 -

17 -# Mixer sample format from pygame.mixer.get_init() -> (array t -ypecode, full scale, value for silence)

18 -SAMPLE_FORMATS = {

19 - -16: ("h", 32767, 0), # signed 16-bit - pygame's def -ault

20 - 16: ("H", 32767, 32768), # unsigned 16-bit

21 - -8: ("b", 127, 0), # signed 8-bit

22 - 8: ("B", 127, 128), # unsigned 8-bit

23 -}

24 -

25 -

26 -class SoundManager:

*27 - def **init**(self):*

28 - """Build all sounds once. If there is no usable audio -device, every

*29 - play_* method quietly does nothing."""

30 - self.fire = None

31 - self.enemy_destroyed = None

32 - self.game_over = None

33 -

34 - try:

35 - if not pygame.mixer.get_init():

36 - pygame.mixer.init()

37 - self.frequency, self.format, self.channels = pygam -e.mixer.get_init()

38 - if self.format not in SAMPLE_FORMATS:

39 - return # unusual format we don't generate - s -tay silent

40 -

… 179 more lines (ctrl+o to expand) Updated tests\test_sound.py (+159 -151) 1 -"*Headless sound tests for Task 4*

(SDL_AUDIODRIVER=dummy - no -thing is heard).

2 -

3 -Run from the project root:

4 - python -m unittest discover -s tests -v

5 -"

6 -import array

7 -import os

8 -import unittest

9 -from unittest import mock

10 -

11 -os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

12 -os.environ.setdefault("SDL_AUDIODRIVER", "dummy")

13 -

14 -import pygame

15 -

16 -from game import sound_manager

17 -from game.sound_manager import SoundManager

18 -from game.game_engine import GAME_OVER_INPUT_DELAY

19 -from tests.test_collision import make_engine, bullet_below

20 -from tests.test_game_over import key, hit_player

21 -

22 -SOUNDS = { # name -> (intended seconds, volume constant)

23 - "fire": (0.12, sound_manager.FIRE_VOLUME),

24 - "enemy_destroyed": (0.25, sound_manager.ENEMY_DESTROYED_VO -LUME),

25 - "game_over": (1.2, sound_manager.GAME_OVER_VOLUME),

26 -}

27 -

28 -

29 -class SoundManagerTests(unittest.TestCase):

30 - def tearDown(self):

31 - pygame.quit()

32 -

33 - def test_sounds_built_with_right_length(self):

34 - # Several mixer formats: a wrong channel count or rate - would change the length

35 - for freq, channels in ((44100, 2), (22050, 1), (48000, - 2)):

36 - pygame.mixer.quit()

37 - pygame.mixer.init(frequency=freq, size=-16, channe -ls=channels)

38 - manager = SoundManager()

39 - self.assertTrue(manager.enabled)

40 - for name, (seconds, _) in SOUNDS.items():

… 270 more lines (ctrl+o to expand) … 2 more files changed (timeout 5m)

● Both test commands now run all 35 tests, and **pycache** no longer clutters git status. One last end-to-end run after the fixes:

Ran 1 shell command (ctrl+o to expand)

● The real-audio run passed only 9 of 10 checks this time. I'll look at which one failed before going any further.

Ran 2 shell commands (ctrl+o to expand)

● The flakiness is a bug in my script, not the game—check_fresh checks a frame too late, so legitimate first-frame enemy fire (especially on Hard) triggers false failures, matching the observed tally. I'll relax the check to allow a first-frame bullet and rerun the 40 trials.

Ran 1 shell command (ctrl+o to expand)

● 40/40 now. Since the scenario script had a randomness flaw, I'll also run the real unit suite 5 times to rule out the same problem there.

Ran 1 shell command (ctrl+o to expand)

● Final review

I found no regressions across the four tasks. The pass did turn up three small problems, all now fixed, plus a few known limits I've left as they are on purpose.

- Tests: all 35 pass with both python -m unittest and the documented discover command, stable across 5 runs.
- Scenarios: I added an end-to-end script that drives the real main.main() loop. It passed 10/10 checks with the real audio device (exit via Esc) and with audio forced to fail (exit via the window's close button), and 40 repeated runs passed.

I re-read every changed file from disk: game_engine.py (46_space-invaders/game/game_engine.py), sound_manager.py (46_space-invaders/game/sound_manager.py), main.py (46_space-invaders/main.py) and all four test files.

1. Scenario walkthrough

The scenario script is in my scratchpad, not the repo. Each check ran through the real main loop:

Scenario	What was checked	Result
Space pressed twice per frame for 60 frames	Firing rapidly	gives exactly 4 bullets and 4 fire sounds	□	(the 15-frame cooldown holds)
Two bullets on	two enemies,	Both die, score +2, 2 destroyed sounds	□	one frame
Getting hit	Game Over, "You were hit", one game-over	□	sound, score frozen
Keys during the	Space, 1 and Esc mashed: ignored, no extra	□	0.5 s delay	sounds
Enemies	Correct reason, game-over sound plays again	reaching the	(re-armed after the restart)	□	bottom
Restart on	Fresh game each time: score 0, 32 enemies,	Easy, Hard,	no player bullets, that level's speed and	□	Medium	fire rate. Exactly 4 game-over sounds over 4	deaths
Exit from the	Game Over	Esc posts QUIT and main() returns normally	□	screen
Closing the	QUIT on the Game Over screen and main()	□	window	returns normally
No audio device	mixer=None, sounds disabled, every scenario	□	above still passes

On the first runs the script failed about 1 time in 7. The cause was my script, not the game: it checked for "no enemy bullets" one frame after a restart. On Hard (5% chance per frame) an enemy can legitimately fire on that first frame. I relaxed the check to allow one first-frame shot, and 40 out of 40 runs then passed. The unit tests don't have this problem, because they check straight after reset() or use fixed random seeds.

2. Problems found and fixed

3. python -m unittest silently ran 0 tests and printed "NO TESTS RAN", because tests/ wasn't a package. A classmate running the obvious command would think nothing was tested. Fix: an empty tests/**init**.py. Both commands now run all 35 tests.

4. Sound didn't handle a pygame build without its mixer. On such builds (some Linux distro packages), any pygame.mixer call raises NotImplementedError, not pygame.error, so the game would crash at startup. Fix: SoundManager now catches (pygame.error, NotImplementedError), with one new test.

5. No .gitignore. Python 3.13 **pycache** files would have gone into your commit. Fix: .gitignore with **pycache**/ and *.pyc.

Known limits I left alone on purpose:

- The starter repo already tracks four Python 3.12 .pyc files. .gitignore doesn't untrack them. If you want them gone, run git rm -r --cached game/**pycache** before committing; that's your call.
- Win screen: the title still says "GAME OVER" (with "You win - …" underneath), and it plays the same "wah".
- Game-over sound: up to 0.7 s of the "wah" can play into a new game if you restart quickly.
- "Bottom of the screen": the game ends when invaders reach the ship's height (the original behaviour), not the literal bottom edge.

3. Summary of what changed, per file

game/game_engine.py (46_space-invaders/game/game_engine.py)

- Task 1: the collision loop no longer removes items from the list it's looping over, which skipped every other bullet. Each loop builds a remaining list instead. Collisions are tested against the area each bullet swept this frame (_move_and_sweep). A fast bullet hits the nearest enemy (_first_enemy_hit). Off-screen bullets are removed after collisions.
- Task 2: the console print and the _game_over_logged hack are gone. _end_game(reason) is now the single way into Game Over. Movement and shooting stop. A dark see-through overlay shows "GAME OVER", the final score and the reason, and keys are ignored for 30 frames.
- Task 3: reset(difficulty) rebuilds all per-game state, and **init** calls it. A DIFFICULTIES table sets Easy, Medium and Hard enemy speed and total shots per second. Enemy fire is now at most one shot per frame from a random enemy, so the rate stays constant: about 2 shots/s on Medium, down from about 19. On the Game Over screen, 1/2/3 restart and Q/Esc post pygame.QUIT. The difficulty shows in the top-right. Clearing the grid ends the game with "You win".
- Task 4: a SoundManager is built once, and the three sounds are hooked to the bullet being created, each kill, and _end_game.

game/sound_manager.py (46_space-invaders/game/sound_manager.py) (new): builds the three sounds once from 16-bit samples generated with array. They match the mixer's real rate and channel count, with fades at both ends and volume constants kept below full scale. Every play_* call does nothing if there's no audio device or mixer. It also guards against a pygame 2.6 crash when a sound is played after the mixer has shut down.

main.py (46_space-invaders/main.py): imports FPS from the engine, so the loop rate and the fire-rate maths can't drift apart. Nothing else changed: exiting from the Game Over screen arrives as a normal pygame.QUIT.

tests/ (46_space-invaders/tests/) (new): 35 headless tests: collision (8), Game Over (8), replay (8) and sound (11). The collision tests fail on the original code. The replay tests check that a restarted engine matches a brand-new one, attribute by attribute.

.gitignore (46_space-invaders/.gitignore) (new): ignores Python bytecode.

4. Checklist for the 10-second "after" video

Before recording:

- [ ] Record system audio as well as the screen (e.g. Win+G Game Bar), and turn the volume on.
- [ ] Run python main.py. It starts on Medium.

The 10 seconds:

1. 0–3 s, Task 1 and fire/kill sounds: fire rapidly into the grid while moving. Every hit registers, with a pew per shot and a crunch per kill.
2. 3–5 s, Task 2 and game-over sound: step under an enemy bullet. The screen dims, "GAME OVER" appears with the final score and "You were hit", and the "wah" plays once.
3. 5–6 s: pause half a second until the "Play again / 1 2 3 / Q Esc" options appear.
4. 6–9 s, Task 3: press 3. A fresh game starts, the top-right says "Hard", and the grid is visibly faster with heavier fire.
5. 9–10 s, if time allows: fire a couple of shots on Hard to show the reset (score back to 0, firing works straight away).

Optional, if you have spare seconds or a second clip: die on Hard and press Esc to show the clean exit.

5. Commit message

Fix bullet collisions; add Game Over screen, replay difficulty, sounds

- Collision (Task 1): stop mutating player_bullets while iterating it, which skipped every other bullet; test the swept path of each bullet so fast bullets can't tunnel; hit the nearest enemy first; remove off-screen bullets after collisions.
- Game Over (Task 2): replace the console print with an overlay showing final score and reason; single _end_game() transition; freeze input; ignore keys for 0.5 s so mashing Space can't dismiss it.
- Replay (Task 3): reset(difficulty) rebuilds all per-game state; Easy/ Medium/Hard table for enemy speed and total shots per second (~2/s on Medium instead of ~19/s); 1/2/3 restart, Q/Esc posts pygame.QUIT; show difficulty in the HUD; clearing the grid ends the game.
- Sound (Task 4): SoundManager generates fire, enemy-destroyed and game-over sounds with array + pygame (no assets, no numpy), matched to the mixer format; silent no-op without an audio device.
- Add 35 headless unittest tests and a .gitignore for bytecode.

Co-Authored-By: Claude Opus 5.5 noreply@anthropic.com (mailto:noreply@anthropic.com)

I haven't committed anything. Your working tree has the changes ready (git add -A && git commit with the message above). If you haven't recorded the "before" video yet, run git stash -u to get the original game back, record it, then git stash pop.